

Lecture 6: **NP**-completeness

Dr. George Mertzios

`george.mertzios@durham.ac.uk`

Room 2066, MCS Building
Tel: 42429

Polynomial-time Reductions

Problem X **polynomially reduces to** problem Y if an arbitrary instance of problem X can be:

- transformed to an instance of problem Y in a polynomial number of steps, and then
- solved using a polynomial number of calls to an oracle that solve problem Y .

Notation: X is polynomial-time reducible to Y is written $X \leq Y$,

If $X \leq Y$ and $Y \leq X$, then X and Y are **equivalent** (with respect to polynomial reductions).

Example

Recall from Lecture 3:

Theorem

INDEPENDENT SET *is equivalent to* VERTEX COVER.

So INDEPENDENT SET can be solved with access to an oracle that can solve VERTEX COVER. Thus

$$\text{INDEPENDENT SET} \leq \text{VERTEX COVER}$$

As the problems are equivalent we also have:

$$\text{VERTEX COVER} \leq \text{INDEPENDENT SET}$$

NP-completeness proofs

To prove that a problem Π is **NP**-complete we now just have to perform two steps:

- 1 Show that Π belongs to **NP**
- 2 Find a known **NP**-complete problem Π' and show $\Pi' \leq \Pi$

Thousands of problems have now been shown to be **NP**-complete (for an early survey see Garey and Johnson).

Note: If we can complete Step 2 but not Step 1, then we say that Π is **NP-hard**

Three proof techniques

Three general methods to prove that an **NP**-complete problem Π' is polynomial-time reducible to a problem Π :

- Restriction
 - ▶ Show that Π' is a subproblem of Π
- Local replacement
 - ▶ Show that every basic unit in an instance of Π' can be replaced by a different structure in a uniform way to obtain an instance of Π .
- Component design
 - ▶ Show that the constituents of an instance of Π can be used to “design” components that can be combined to “realize” (or to “encode”) instances of Π' .

NP-completeness of VERTEX COVER

VERTEX COVER

Instance: A graph $G = (V, E)$, and a natural number k .

Question: Is there a set $W \subseteq V$, with $|W| \leq k$, such that for each edge $(i, j) \in E$,

$$\{i, j\} \cap W \neq \emptyset$$

The problem VERTEX COVER is in **NP** (the list of vertices in the cover is the certificate)

To show that VERTEX COVER is **NP**-complete we shall reduce SATISFIABILITY to VERTEX COVER

NP-completeness of VERTEX COVER

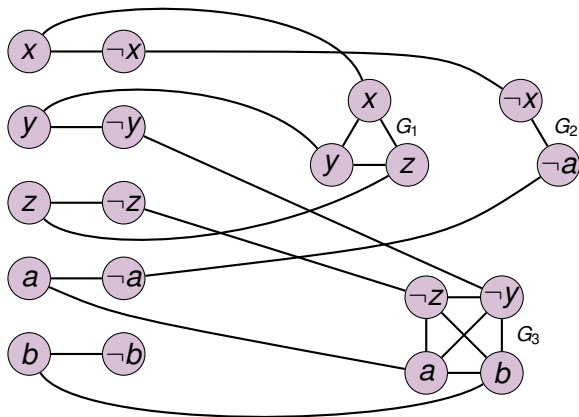
Given a formula f with n variables and clauses $C_1, C_2, \dots, C_m$

- For each variable x create two adjacent vertices x^t and x^f to represent the *literals* x and $\neg x$
- For each clause C_j of size n_j create a complete subgraph G_j with vertices connected to corresponding literals
- Set $k = n + \sum_{j=1}^m (n_j - 1)$

This construction can be carried out in polynomial time

Example construction

$$\underbrace{(x \vee y \vee z)}_{C_1} \wedge \underbrace{(\neg x \vee \neg a)}_{C_2} \wedge \underbrace{(a \vee b \vee \neg y \vee \neg z)}_{C_3}$$



$$k \leq 5k - 252 + 13 = 5k - 239$$

NP-completeness of VERTEX COVER

Theorem

There exists a truth assignment that satisfies the formula f if and only if there exists a vertex cover of the constructed graph with size at most k .

Sketch of the proof: direction ($\Rightarrow$)

- At least one of each pair (x^f, x^t) must be in the cover
- At least $n_j - 1$ vertices from each complete graph G_j must be in the cover
- If the formula is satisfiable, then choose the cover by choosing each literal assigned TRUE plus all but one vertex in each G_j . (Omit a vertex which is connected to a satisfied literal.)

NP-completeness of VERTEX COVER

Theorem

There exists a truth assignment that satisfies the formula f if and only if there exists a vertex cover of the constructed graph with size at most k .

Sketch of the proof: direction ($\Leftarrow$)

- Conversely, if a vertex cover exists, assign each Boolean variable according to whether x^t (TRUE) or x^f (FALSE) is in M .

By the choice of k , there must be exactly one vertex in each clique which is not in M . This vertex must be adjacent to a literal-vertex in M , hence the clause is satisfied.

Therefore, since SATISFIABILITY is **NP**-complete, it follows that:

Theorem

VERTEX COVER is **NP**-complete.

NP-completeness of CLIQUE

CLIQUE

Instance: A finite graph $G = (V, E)$ and an integer k .

Question: Does G have a clique of size k ?

- Recall (see Lecture 3) that there is a straightforward reduction from VERTEX COVER to CLIQUE: a set of vertices W is a vertex cover in G if and only if $V - W$ is a clique in the complement of G .
- Can we also find a reduction from SATISFIABILITY to CLIQUE?

Hitting Set

HITTING SET

Instance: Collection C of subsets of a set S and a positive integer k .

Question: Does S contain a *hitting set* for C of size k or less?

i.e.: a subset $S' \subseteq S$ with $|S'| \leq k$ such that S' contains at least one element from each subset from C ?

HITTING SET is in **NP** (the subset S' is the certificate)

To show that HITTING SET is **NP**-complete: restrict it to instances with $|c| = 2$ for all $c \in C$, i.e. every set c in C has 2 elements.

Which problem do you get then? **Vertex Cover !**

Main idea:

- S is the set of vertices of a graph G
 - the collection C of subsets of S is the set of edges of G
- ⇒ a hitting set of size k is a set of k vertices (from S) which “hit” (i.e. cover) all edges (in C)

3-SATISFIABILITY

To show that 3-SATISFIABILITY is **NP**-complete we reduce SATISFIABILITY to 3-SATISFIABILITY.

Proof idea:

- Replace every clause

$$C = x_1 \vee x_2 \vee \dots \vee x_k$$

with $k > 3$ by:

$$C' = (x_1 \vee x_2 \vee y_1) \wedge (\neg y_1 \vee x_3 \vee y_2) \wedge \dots \wedge (\neg y_{k-3} \vee x_{k-1} \vee x_k)$$

- C is satisfiable if and only if C' is, since at least one of the literals other than the y 's must be TRUE .

NP-completeness of CLIQUE

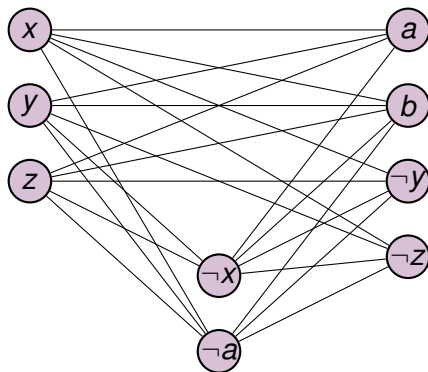
- The problem CLIQUE is in **NP** (the list of vertices in the clique is the certificate)
- To show that CLIQUE is **NP**-complete we shall reduce SATISFIABILITY to CLIQUE
- Given a formula f with clauses $C_1, C_2, \dots, C_k$
 - ▶ For each literal x in clause C_j create a vertex
 - ▶ Put an edge between every pair of vertices from distinct clauses, unless they are **incompatible** (i.e. x and $\neg x$)

This construction can be carried out in polynomial time

- The resulting graph has a clique of size k if and only if f is satisfiable (assign the value TRUE to every literal occurring in the clique)

Example construction

$$\underbrace{(x \vee y \vee z)}_{C_1} \wedge \underbrace{(\neg x \vee \neg a)}_{C_2} \wedge \underbrace{(a \vee b \vee \neg y \vee \neg z)}_{C_3}$$



$$k = 3$$

Additional slides

NP-completeness of SUBSET SUM

SUBSET SUM

Instance: A set of positive integers $S = \{n_1, n_2, \dots, n_k\}$, and a “target” integer t .

Question: Is there a subset $T \subseteq S$, such that $\sum_{i \in T} n_i = t$?

SUBSET SUM is in **NP** (the set T is the certificate)

To show that SUBSET SUM is **NP**-complete we will reduce SATISFIABILITY to SUBSET SUM.

NP-completeness of SUBSET SUM

Given a formula f with clauses $C_1, C_2, \dots, C_k$

- For each **variable** x create **two** appropriate **integers** $\{n_{xt}, n_{xf}\}$
- For each **clause** C_j of length l_j , create $l_j - 1$ **integers** $\{m_{j1}, m_{j2}, \dots\}$
- Choose the **“target” integer** T such that T must contain:
 - ▶ **exactly one** of each **pair** $\{n_{xt}, n_{xf}\}$
⇒ this indicates that every variable is either TRUE or FALSE
 - ▶ **at least one** from each **tuple** $\{m_{j1}, m_{j2}, \dots\}$
⇒ this indicates that every clause is satisfiable

This construction can be carried out in polynomial time.

However, exact details about how we compute the integers $\{n_{xt}, n_{xf}\}$, $\{m_{j1}, m_{j2}, \dots\}$, and t must be specified.

NP-completeness of SUBSET SUM

We prove that if a subset T can be found, then the formula is satisfied (the proof of the reverse is left as an exercise).

- T includes exactly one of $\{n_{xt}, n_{xf}\}$ for each variable x ;
use this to decide whether each variable is TRUE or FALSE
 - ▶ i.e. the set T in the solution of the derived SUBSET SUM instance contains the information about which variables x are TRUE and which are FALSE in the initial SATISFIABILITY instance
- for each clause C_i , T contains an integer of $\{n_{xt}, n_{xf}\}$ that represents a literal in C_i ;
- thus each clause (and so also the whole formula) is satisfied.

Example Construction

$$(x \vee y \vee z) \wedge (\neg x \vee \neg a) \wedge (a \vee b \vee \neg y \vee \neg z)$$

		x	y	z	a	b	C_1	C_2	C_3
n_{xt}	=	1	0	0	0	0	1	0	0
n_{xf}	=	1	0	0	0	0	0	1	0
n_{yt}	=		1	0	0	0	1	0	0
n_{yf}	=		1	0	0	0	0	0	1
n_{zt}	=			1	0	0	1	0	0
n_{zf}	=			1	0	0	0	0	1
n_{at}	=				1	0	0	0	1
n_{af}	=				1	0	0	1	0
n_{bt}	=					1	0	0	1
n_{bf}	=					1	0	0	0
m_{11}	=						1	0	0
m_{12}	=						1	0	0
m_{21}	=						0	1	0
m_{31}	=						0	0	1
m_{32}	=						0	0	1
m_{33}	=						0	0	1
T	=	1	1	1	1	1	3	2	4